

Rapport sur les langages de programmation

Application : Fad Agency OS

Projet	Fad Agency OS
Type	Application desktop + API backend + base PostgreSQL
Objectif du rapport	Identifier les langages et technologies utilisés, leur rôle et leur importance pour la suite du développement.
Date	24/06/2026

Résumé exécutif

Fad Agency OS est une application moderne construite principalement en TypeScript. Le frontend desktop utilise React, Vite et Tauri. Le backend utilise NestJS avec Prisma et PostgreSQL. Rust est présent pour la couche native Tauri, tandis que Tailwind/CSS gère le design et les thèmes.

Conclusion rapide : pour développer efficacement cette application, la compétence numéro 1 est TypeScript full-stack, avec une bonne maîtrise de React, NestJS, Prisma, PostgreSQL et Tailwind CSS.

1. Architecture générale

L'application est organisée comme un monorepo avec pnpm et Turbo. Le fichier racine indique notamment pnpm, Node >=18.18.0, ainsi que les scripts dev, build, lint, test, desktop et API.

Couche	Technologies principales	Langage dominant	Rôle
Frontend Desktop	React, Vite, Tauri, TanStack Query, Zustand, i18next	TypeScript / TSX	Interface utilisateur et expérience desktop
Backend API	NestJS, Prisma Client, JWT, Zod, Vitest	TypeScript	API métier, sécurité, permissions et logique data
Native Desktop	Tauri 2, Serde, Keyring	Rust	Fenêtre native Windows et intégration système
Database	PostgreSQL, Prisma migrations	Prisma Schema / SQL	Stockage des clients, paiements, tâches, équipe, paramètres
Design	Tailwind CSS, CSS variables, PostCSS	CSS	Thème, layout, composants, RTL

2. Langage principal : TypeScript

TypeScript est le langage central de Fad Agency OS. Il est utilisé côté frontend et côté backend. Cela permet d'avoir une logique typée, plus robuste, et plus facile à maintenir.

Où TypeScript est utilisé

- Frontend React : pages, composants, hooks, formulaires, stores, routes.
- Backend NestJS : modules, controllers, services, DTOs, guards, tests.
- Validation et typage : Zod, DTOs, types de réponse API, tests Vitest.
- Build et qualité : tsc, ESLint, Vitest, Vite, Nest build.

Importance : très élevée. Toute évolution fonctionnelle sérieuse de l'application passe par TypeScript.

3. Frontend : React / TSX

Le frontend de l'application desktop est construit avec React 18 et Vite. Les fichiers d'interface sont majoritairement en TSX, c'est-à-dire TypeScript avec JSX.

Élément	Technologie	Rôle
UI	React + TSX	Pages, composants, formulaires, tableaux, modales
Routing	React Router	Navigation entre Dashboard, Clients, Tasks, Settings, etc.
Data fetching	TanStack Query	Appels API, cache, refetch, états loading/error
State	Zustand	État local/global comme session, settings ou toast
Validation	Zod + React Hook Form	Validation formulaires côté frontend
Charts	Recharts	Graphiques Dashboard
i18n	i18next + react-i18next	FR/EN/AR et RTL

4. Backend : TypeScript avec NestJS

Le backend est développé avec NestJS, un framework Node.js orienté architecture modulaire. Il gère les routes API, l'authentification, les permissions, et les opérations métier.

Module / Fonction	Rôle backend
Auth / JWT	Connexion, token d'accès, token refresh, utilisateur courant
RBAC	Rôles, permissions, contrôle d'accès
Clients	CRUD client, données agence, social links en Phase 7.3B
Finance	Payments, wallets, ad spend, expenses
Operations	Tasks, team members, notifications
Reports	KPIs dashboard et indicateurs

Importance : très élevée. Pour préparer l'hébergement et une version SaaS, le backend doit devenir encore plus solide : logs, sécurité, jobs, exports, permissions et tests.

5. Rust : couche native Tauri

Rust est utilisé par Tauri pour transformer l'application web React en application desktop native. Dans ce projet, Rust ne porte pas la logique métier principale ; il sert surtout à l'enveloppe desktop, aux commandes système et au stockage sécurisé des tokens.

Fichier / élément	Ce que cela indique
Cargo.toml	Package Rust fad-agency-os, édition Rust 2021
tauri = 2	Shell desktop natif
serde / serde_json	Sérialisation des données entre frontend et native shell
keyring	Stockage sécurisé local des tokens dans l'OS

Importance : moyenne. Il faut surtout que l'environnement Rust/MSVC soit stable pour builder et lancer l'application Tauri.

6. CSS / Tailwind CSS

Le design de l'application repose sur CSS, Tailwind CSS et des design tokens. C'est ce qui gère le dark mode, le light mode, les couleurs d'accent, la sidebar, la topbar, les cards, les tableaux et le support RTL pour l'arabe.

- Tailwind CSS : classes utilitaires pour créer rapidement l'interface.
- CSS variables : tokens de couleurs, rayons, ombres, thèmes.
- PostCSS / Autoprefixer : compatibilité CSS.
- RTL : adaptation visuelle quand l'arabe est sélectionné.

7. Prisma Schema et SQL / PostgreSQL

Prisma Schema décrit les modèles de données et génère le client TypeScript. PostgreSQL est la base réelle utilisée par l'application.

Élément	Langage / format	Rôle
schema.prisma	Prisma Schema	Définir User, Client, Payment, Wallet, Task, TeamMember, Setting, etc.
Migrations	SQL généré	Modifier la structure de PostgreSQL de façon contrôlée
PostgreSQL	SQL	Stocker les données réelles de l'agence
Prisma Client	TypeScript généré	Accéder à la DB dans NestJS

Importance : très élevée. Toute fonctionnalité qui demande de nouvelles données - par exemple liens sociaux client, historique abonnement, statut paiement dépense - passe par Prisma et PostgreSQL.

8. JSON et configuration

JSON est utilisé pour les fichiers package.json, la configuration de dépendances, scripts, i18n et divers réglages. Ce n'est pas un langage métier, mais il est nécessaire pour maintenir le projet.

Fichier / usage	Rôle
package.json racine	Scripts dev/build/lint/test, pnpm, Node engine
apps/desktop/package.json	Dépendances frontend : React, Vite, Tauri, Tailwind, Vitest
apps/api/package.json	Dépendances backend : NestJS, Prisma, JWT, Vitest
i18n resources	Clés de traduction FR/EN/AR

9. PowerShell

PowerShell n'est pas le langage de l'application, mais il est utilisé pour lancer certains scripts Windows, notamment les scripts MSVC/Tauri.

- dev-tauri-msvc.ps1 : lancer Tauri dev avec l'environnement MSVC.
- build-tauri-msvc.ps1 : builder l'application Windows.
- msvc-env.ps1 : charger Visual Studio Build Tools / link.exe / cl.exe.

10. Classement par importance

Rang	Langage / technologie	Importance	Pourquoi
1	TypeScript	Très élevée	Langage principal frontend + backend
2	React / TSX	Très élevée	Toutes les pages et composants UI
3	NestJS	Très élevée	API métier, auth, RBAC, services
4	Prisma Schema	Très élevée	Modèles DB et migrations
5	SQL / PostgreSQL	Très élevée	Données réelles et production
6	CSS / Tailwind	Élevée	Design, thèmes, responsive, RTL
7	Rust	Moyenne	Shell Tauri et build desktop
8	JSON	Moyenne	Configuration, scripts, i18n
9	PowerShell	Moyenne	Scripts Windows de développement

11. Recommandations pour continuer le développement

- Prioriser TypeScript full-stack : c'est la compétence la plus importante pour ce projet.
- Garder les changements DB additive-only : migrations Prisma sans reset ni suppression de données.
- Renforcer les tests Vitest côté desktop et API avant chaque merge.
- Stabiliser les fonctionnalités métier avant l'hébergement : clients, tâches équipe, dépenses, abonnements, notifications.
- Garder Rust/Tauri simple : éviter de déplacer de la logique métier dans Rust sauf besoin système réel.

12. Conclusion

Fad Agency OS est techniquement basé sur une stack moderne et cohérente : TypeScript pour la logique, React pour l'interface, NestJS pour l'API, Prisma/PostgreSQL pour la donnée, Tailwind pour le design et Rust/Tauri pour le desktop. Cette architecture est adaptée à une future version professionnelle destinée à une agence marketing ou e-commerce.

Sources techniques consultées

Source	Information utilisée
package.json racine	pnpm, Node engine, scripts dev/build/lint/test
apps/desktop/package.json	React, Vite, Tauri, Tailwind, i18next, Vitest, TypeScript
apps/api/package.json	NestJS, Prisma, JWT, Vitest, TypeScript
apps/desktop/src-tauri/Cargo.toml	Rust, Tauri 2, Serde, Keyring
apps/api/prisma/schema.prisma	Prisma/PostgreSQL et modèles de données